

FIXERUPPER

FROM STORE TO DOOR

PROJECT DOCUMENTATION

PHP, MYSQL, RESPONSIVE STOREFRONT AND CART INTERFACE

I. APPLICATION OVERVIEW

The storefront is rendered by ``index.php``. It loads active products from the database, attaches gallery images and specification rows, and exposes the resulting data to ``product-modal.js``. Each product card contains the main image, title, summary, price, and a More Info action. The modal presents a larger image stage, navigation dots, product specifications, and an Add to Cart button. Feedback such as "Added to cart" and "This product is already in the cart" is

displayed as centered animated text, avoiding layout shifts.

The cart is rendered by ``cart.php`` from ``$_SESSION['cart']``. The cart controller supports plus/minus buttons and direct numeric input. Invalid input is sanitized, values are capped at 99, and a quantity of zero opens a confirmation state instead of deleting the row immediately. Pricing remains server-authoritative: JavaScript submits the requested quantity, while PHP reloads the product price from the database and returns the formatted line total.

II. FRONTEND AND RESPONSIVE OPTIMIZATION

The CSS is centralized in ``assets/css/style.css``. The site uses a shared dark background, gradient cards, metallic text, muted advertising logos, and precise divider effects. The cart has two explicit body states: ``cart-body--empty`` and ``cart-body--filled``. Empty cart pages retain the homepage rhythm, including the top stripe and broad background gradient. Filled carts use a table-specific visual layer so the header, item rows,

dotted divider, and advertising block do not create duplicated stripes or stacked gradients.

Mobile optimization is handled through breakpoints at 1020px, 760px, and 640px. Navigation stacks into centered rows, product cards collapse to a single column, and cart rows become compact two-line entries: image on the left, title and price in the middle, quantity controls on the right. Fixed track widths, stable quantity capsule dimensions, hidden overflow, and row-specific gradients keep the mobile layout readable during browser zoom and on narrow screens.

III. DATABASE DESIGN

The database is defined in ``database/fixerupper.sql`` and uses InnoDB with ``utf8mb4``. ``products`` is the central storefront table and stores the slug, display name, short description, price, main image, and active flag. ``product_images`` stores ordered gallery images, while ``product_specs`` stores modal specification rows. Both tables reference ``products(id)`` and use cascading deletes, so dependent content is removed when a product is removed.

The checkout-ready part of the schema contains ``users``, ``orders``, and ``order_items``. One user may have many orders. One order may contain many order items. Each order item references both an order and a product, preserving quantity and unit price for future payment flow. Product deletion is restricted for saved order lines, protecting historical order data. Unique indexes prevent duplicate gallery positions, duplicate specification positions, and duplicate product rows inside the same order.

IV. DATA AND SEED STRATEGY

The SQL seed creates six active PC products, one gallery image per product, and full modal specifications. Inserts use `ON DUPLICATE KEY UPDATE`, making the seed idempotent. This is

useful for local development because the database can be re-imported without duplicating products, images, or specification rows. The current cart remains session-based, so order tables are prepared for future work but are not yet written by the UI.

V. DATABASE RELATIONSHIP SCREENSHOT

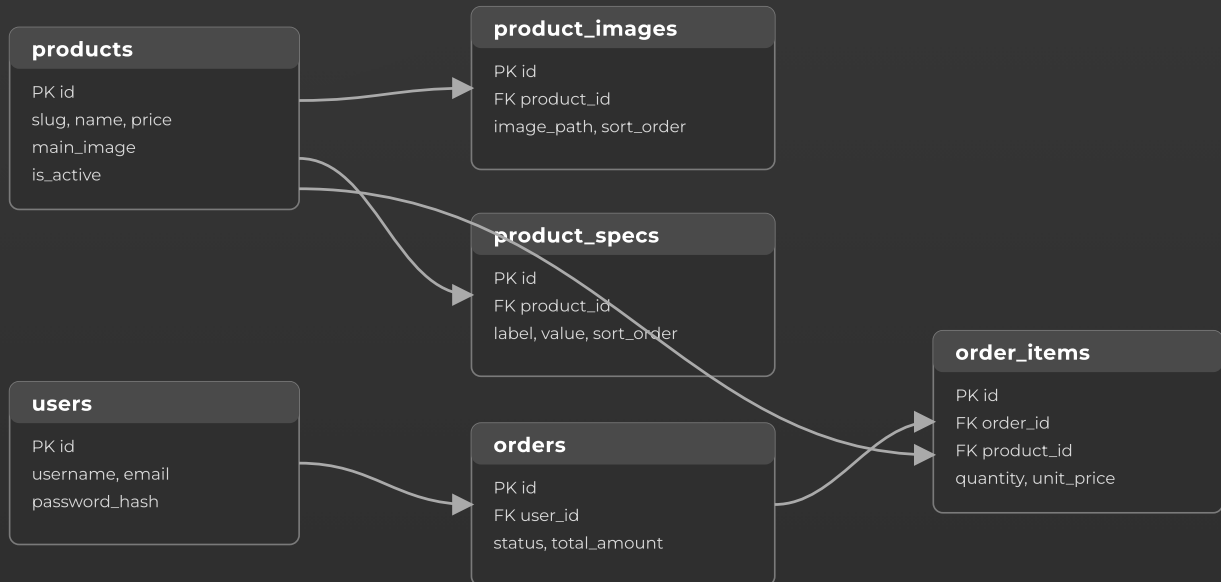


Figure 1. Database relationship diagram showing primary entities and foreign-key paths.

VI. DESKTOP INTERFACE SCREENSHOTS

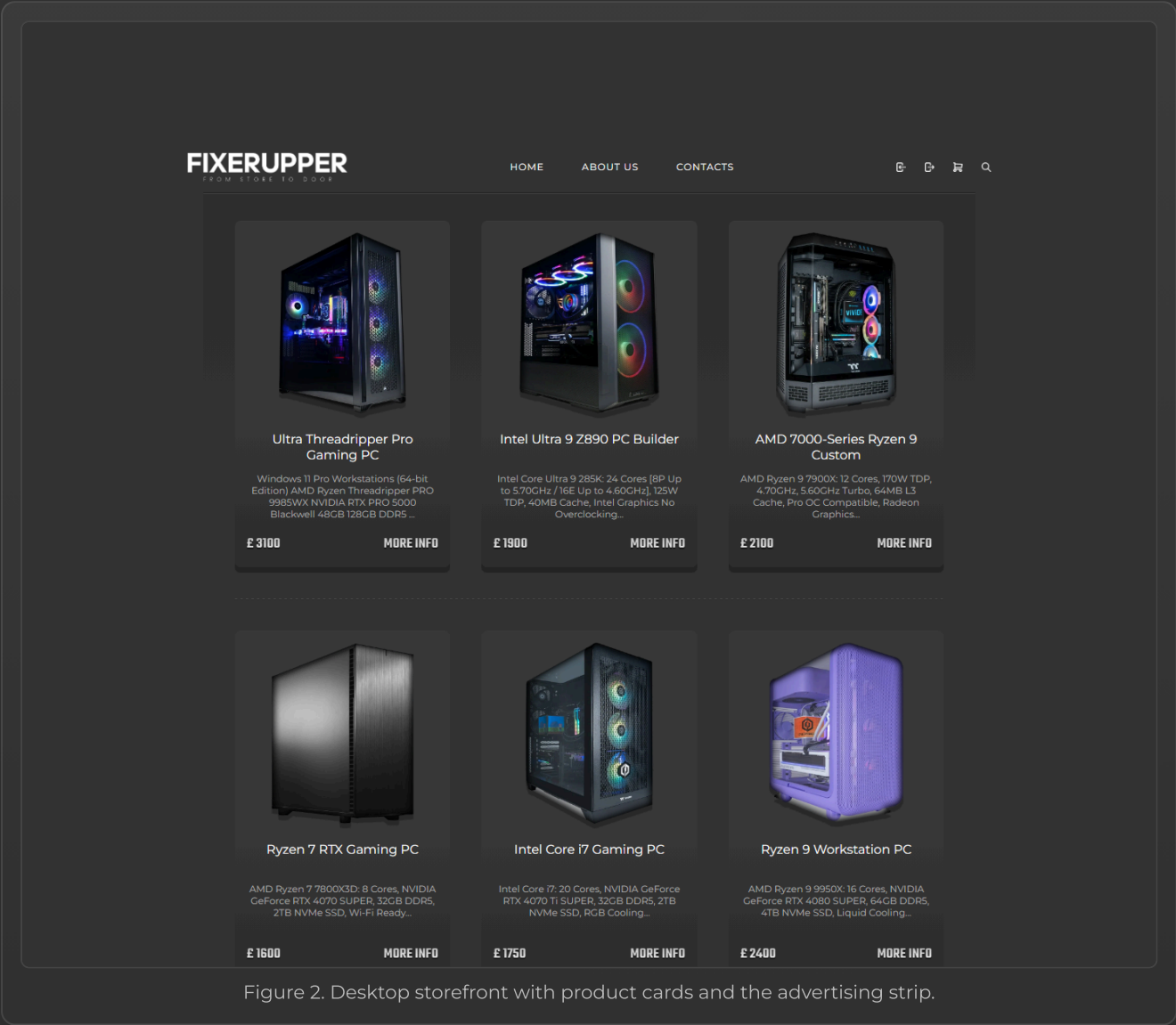


Figure 2. Desktop storefront with product cards and the advertising strip.



Figure 3. Desktop More Info modal with gallery and specification panel.

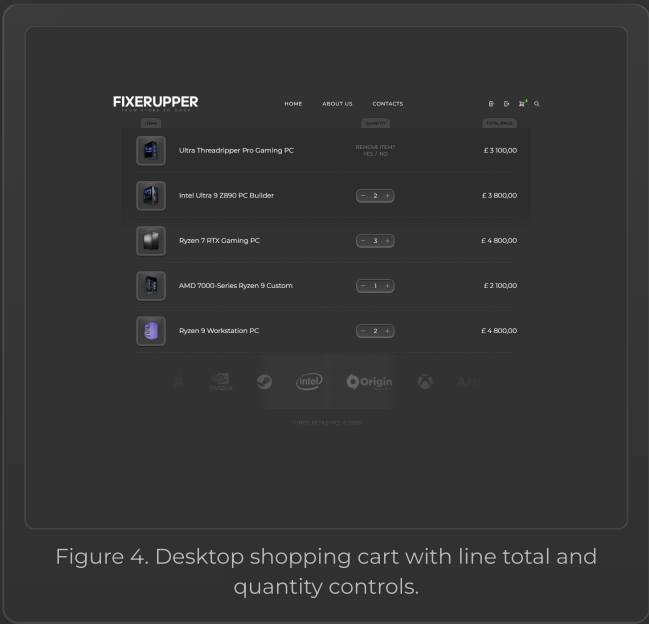


Figure 4. Desktop shopping cart with line total and quantity controls.

VII. MOBILE INTERFACE SCREENSHOTS

The mobile captures demonstrate the optimized single-column storefront, stacked modal presentation, and compact cart row. The layout keeps stable control sizes and uses row-scoped backgrounds to avoid duplicated gradients or stripes.

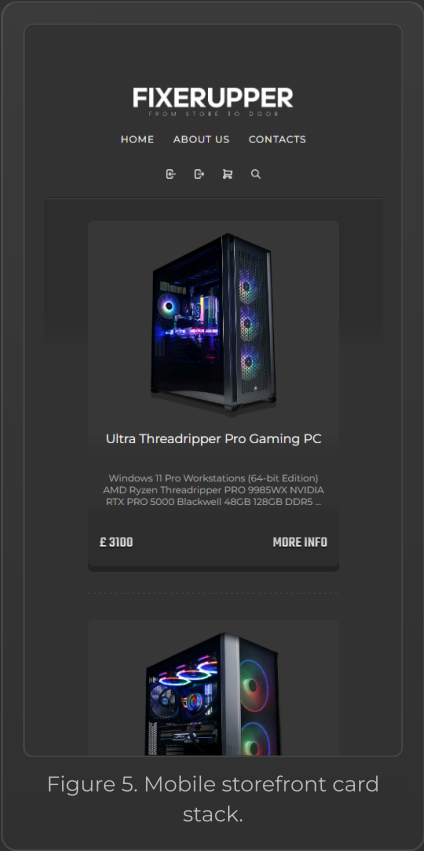


Figure 5. Mobile storefront card stack.

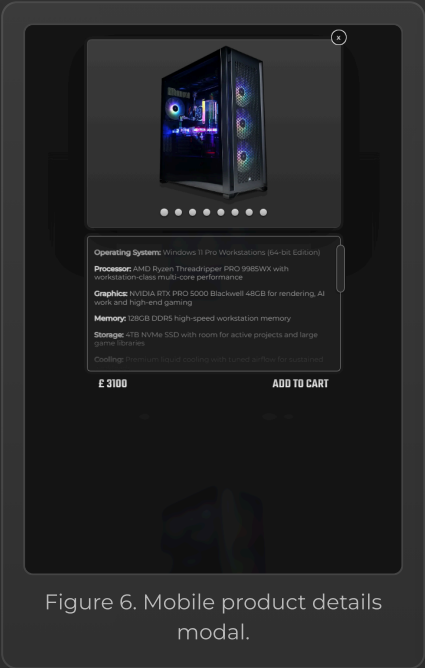


Figure 6. Mobile product details modal.

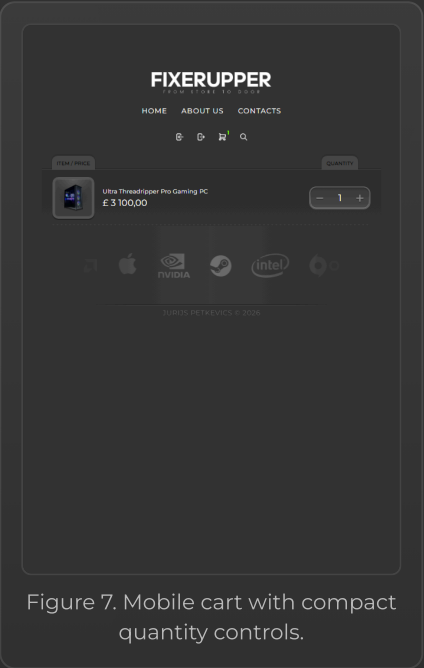


Figure 7. Mobile cart with compact quantity controls.

VIII. CONCLUSION

FIXERUPPER now provides a coherent storefront and cart workflow across desktop and mobile breakpoints. The project keeps the implementation intentionally direct: PHP renders the main pages, MySQL stores product data and future checkout structures, JavaScript handles interactive cart and modal behavior, and CSS defines the visual system. The database is ready for a later checkout phase, while the current interface already supports product browsing, modal inspection, duplicate-cart feedback, direct quantity editing, removal confirmation, and responsive presentation without duplicate gradients or stripe artifacts.